

Automatic maintenance related procedures

June 26, 2014 5:28 PM

Maint.YourSqlDbA_DoMaint does most of the default maintenance, and calls some of the other procedures below to do it. It have several parameter that are customized

For most people having to manage a database server this is more than enough to get a nice maintenance plan.

For more seasoned DBA sometimes customization of this procedure beyond the ones allowed by initial setup is needed. It can be easily be achieved by modifying its parameters in job steps. For a description of these parameters see: [Maint.YourSQLDbA DoMaint](#)

Parameters can be modified from SQL Management Studio\SQL Agent\YourSqlDbA... Job \Job Step

Job schedule execution can be modified in SQL Management Studio\SQL Agent\YourSqlDbA...Job \JobSchedule.

Email configuration can be changed from SQL Management Studio\Manage\DatabaseMail.

Maint.YourSQLDb _DoMaint

June 20, 2014 5:08 PM

Purpose: Main procedure used to process all regular maintenance on databases. Usually called from a SQL Server Agent job step, it can also be call manually. To customize maintenance you can add tasks calling this procedure.

Parameter name	Parameter description
@oper	SQL Server Agent Operator. Any valid operator name is allowed. By default 'YourSQLDb _Operator'. (nvarchar(200))
@MaintJobName	nvarchar(200) SQL Server Agent job name logged into the maintenance history. Any valid job name is allowed. By default 'YourSQLDb: DoInteg,DoUpdateStats,DoReorg,Full backups'. (nvarchar(200))
@DoInteg	When equal to 1 perform Integrity tests on selected databases. By default 0 (Int)
@DoUpdStats	When equal to 1 perform Update Statistics on selected databases. By default 0 (Int)
@DoReorg	When equal to 1 perform Selective Index Reorganization or Rebuild depending of an internal fragmentation level found on them. When Rebuilding now leave the original FillFactor. By default 0 (Int)
@DoBackup	When equal to 'F'; full database backups are performed, plus an initial log backup. When equal to 'L'; log backups are performed. When equal to 'D' differential backups are done. In case of log backups, @FullBackupPath and @LogBackupPath are ignored, the value used at last full backup performed by Maint.YourSQLDb _DoMaint or Maint.SaveDbOnNewFileSet , is the one used for log backups. When an empty string is supplied no backup is performed. (nvarchar(5))
@FullBackupPath	Location where full database backup files (.Bak) are going to be stored. (Required parameter) . The UNC format is allowed. Example A : 'G:\SQL\Backups' Example B : '\\SQL\Backups' (nvarchar(512))
@LogBackupPath	Location where log database backup file (.Trn) are going to be stored. (Required parameter) . The UNC format is allowed. Example A : 'G:\SQL\Backups' Example B : '\\SQL\SQLBackups' We recommend that you use the same location as the @FullBackupPath parameter. In this way, all of your backup files concerning a database would be shown on the same page in chronological order in Windows Explorer. Nevertheless you can use a different partition if the disk is full. (nvarchar(512))
@TimeStampNamingForBackups	By default YourSqlDb add a timestamp (date and time) to backup file names. When specified with a value of 0 this parameter stops this behavior. You need to be aware that the parameters @FullBkpRetDays and @LogBkpRetDays parameters become ineffective, since total backups and log backups are overwritten at each full maintenance. Log backups still accumulate into the same log backup file during the day no matter the value of the parameter. (Int) New with version 6.2.4.
@FullBkpRetDays	Number of days to retain the preceding full backups. Older backups are deleted from the backup directory. NULL = no cleanup (default). (Int)
@LogBkpRetDays	Number of days to retain the preceding transaction logs backups. Older backups are deleted from the backup directory. NULL = no cleanup (default). (Int)
@NotifyMandatoryFullDbBkpBeforeLogBkp	Apply to log backups only. When equal to 1 an error is reported when a log backup can't be done because full database backup was never performed since database creation or since last restore. When equal to 0 this error is not reported. A value of 0 applies usually well on test databases, restored from production database for which we don't really care about doing log backup. (Int)
@BkpLogsOnSameFile	Log backups storing method. 1 = Add the log backups to the same .trn backup file available since the last full backup. (default). 0 = Create a new backup file for each log backup. (Int)
@SpreadUpdStatRun	The "update statistics" for database tables is spread on the specified number of executions. By default the "update statistics" is spread on a 7 executions cycle. At the end of the cycle all tables in the database will have been treated. (Int)
@SpreadCheckDb	The full DBCC CHECKDB for database spread on the specified number of executions of the main maintenance. By default the "DBCC CheckDb" is spread on a 7 executions cycle. It is not the turn of a given database to be fully DBCC checked, it is checked faster with DBCC CheckDb ... WITH

	Physical_Only and only if its page verification option is set to Checksum. If page verification option is not to "checksum", DBCC CheckDb will always without this option.
@ConsecutiveDaysOfFailedbackupsToPutDbOffline	Maximum number of consecutive days of failed full backups allowed for a database before putting that database (Offline). One successful full backup will reset to zero the counter of consecutive days of failed full backups for that database. By default = 9999. Last resort action to inform database administrators Normally YourSQLDb sends an email to the SQL administrators when there is a full backup failure. However if these emails stop reaching their recipients, there is a need for a last resort mechanism to inform the administrators that the data is not protected. The maintenance parameter @ConsecutiveDaysOfFailedbackupsToPutDbOffline instructs YourSQLDb to put offline a database as a last resort to signal that full backups of a given database have failed for the specified number of consecutive days. When the full backup maintenance job is executed a counter is incremented once a day for each database for which the full backup failed. If this days counter is equal to the maximum number of consecutive failed full backup days allowed by the parameter @ConsecutiveDaysOfFailedbackupsToPutDbOffline, the corresponding database is put offline. The counter is reset to zero when the full backup succeeds for the database. This feature is a last resort to attract attention of a database administrator or application users to a potentially dangerous backup problem that can lead to data loss. Note that the suspension of service can occur only at the end of the full backup maintenance job which is usually in the middle of the night. Important: Putting a database offline implies a temporary suspension of service. You may not want that to happen. Even if no backups are available for the database. You must set a value for this number of days allowed for full backup failures. The number of days to choose depends on the availability level of the database administrators and of the risk tolerance for data loss. When full backups don't work, it's usually because the disk is almost full. Most of the time the log backups continue even when the full backups didn't. This is because log backups need less disk space than full backups. The log backups allow the restoration of a database from the last available full backup. Keep in mind that the greater the number of days gone without full backups or even without transaction logs backups, the riskier it is to lose data in case of a SQL server failure. You have to decide the threshold number of days of full backup failures that will cause the temporary suspension of service for the database. Here are 3 scenarios to choose from 1. If you think that your organisation has fail safe SQL management practices to guarantee that backups are checked every day, you can turn off this behaviour by using a value of 9999 days: @ConsecutiveDaysOfFailedbackupsToPutDbOffline= 9999 Specifying a great number of days for the @ConsecutiveDaysOfFailedbackupsToPutDbOffline parameter will effectively disable it. 2. If you think that there would not be a database administrator available every morning to verify the production of the full backups or to put the databases online in case of a full backup failure, you can use a value of 20 days : @ConsecutiveDaysOfFailedbackupsToPutDbOffline= 20 3. If you think that there will always be a database administrator available every morning to verify the production of the full backups or to put the databases online in case of a full backup failure, then you can set a value of 4 days by running the following command in SQL Management Studio: @ConsecutiveDaysOfFailedbackupsToPutDbOffline= 4 (Int)
@MirrorServer	Name of the mirror SQL instance destination of the automatic restoration of the backups. (Optional parameter). The Mirroring.AddServer procedure must be executed to record this mirror SQL instance, prior to using it here in the @MirrorServer parameter. Example: 'MirrorSQLServ\MirrorSQL2005'

	NULL = don't restore on a mirror SQL instance. Executed tasks when @MirrorServer contains a SQL instance name: • Restore the backups on the mirror SQL instance. The goals for this restoration are to verify the integrity of the backups, and to create duplicates of the databases in « RECOVERING » status. An exception applies when @migrationTestMode=1 The automatic restoration works like this: as soon as YourSqlDbA creates a backup file, it restores it on the mirror SQL instance. If this restoration fails; an error email is sent to the SQL administrator. • Copy the SQL logins to the mirror SQL instance. (Int)
@migrationTestMode	This parameter goes along with @MirrorServer, where mirrorServer is the newer version of SQL on which we want to migrate some databases. It allows to have a recovered working copy of database, without worrying having error messages from YourSqlDbA mirroring feature, saying that databases it has to restore can't be restored, because they are recovered. When enabled (@migrationTestMode = 1), just before restoring full backups, YourSqlDbA first check if the database exists on the mirrorServer. If it is, it just skips the restore. The same applies for log restores. If the database isn't there it just restores again at next backup and brings it online. So to get a newer copy of the database, delete it first. Once done, one does have to wait for next full backups, or do a SaveDbOnNewFileset. Once someone is done with the database it want to test on the new server, just delete databases on the new server and remove @migrationTestMode parameter (It's default value is zero) (int)
@ReplaceSrcBkpPathToMatchingMirrorPath	Does a "string like" replace into backup location uses by source server to get backup location as seen from mirror server. For example suppose backups are put onto a local directory (here "F:\SqlProd\backups\") and this directory is exposed to the mirror server through a share as (here: "\\sqlProd\backups"). When mirror server is instructed to do a restore, it will replace into the whole file path "F:\SqlProd\backups\" by "\\sqlProd\backups". This is necessary since the restore command can't be performed from its local path since while the backup file is local to the source server it isn't for the mirror. Mirror needs a different way to name the backup it want to restore. The same holds true for SQLReport location. <pre>@ReplaceSrcBkpPathToMatchingMirrorPath = ', F:\SqlProd\backups\>\\sqlProd\backups\ F:\SqlReport\backups\>\\sqlReport\backups\ '</pre> can also be written as : <pre>@ReplaceSrcBkpPathToMatchingMirrorPath = ', F:\SqlProd\backups\>\\sqlProd\backups\ F:\SqlReport\backups\>\\sqlReport\backups\ '</pre>
@ReplacePathsInDbFileNames	@ReplacePathsInDbFileNames, will allow "string like" replace of part of path/file names in the databases files to be restored. If omitted databases are restored to default data and log database locations. For example suppose that: source Db files from SQLProd are located under 'D:\SqlProd\Data' for data And 'E:\SqlProd\Logs' for logs and source Db files from SQLReport are located under 'D:\SqlReport\Data' for data And 'E:\SqlReport\Logs' for logs. If you want to put them on the mirror server on different disks and subdirectories let's say 'G:\Mirror\SqlProd\Data' And 'H:\Mirror\SQLProd\Logs' for SQLProd and 'G:\Mirror\SQLReport\Data' And 'H:\Mirror\SQLReport\Logs' for SQLReport, @ReplacePathsInDbFileNames will have to be specified this way: <pre>@ReplacePathsInDbFileNames = ', D:\>G:\SqlMirror\ E:\>H:\SqlMirror\ ', Or @ReplacePathsInDbFileNames = 'D:\>G:\SqlMirror\ E:\>H:\SqlMirror\ '</pre> To restore to same location (let's say they are under D and E drive) use: <pre>@ReplacePathsInDbFileNames = ', D:>D: E:>E: </pre>

@IncDb	One database to select, or many using a database list, or many more using a wildcard database list. When left empty, all databases are selected for maintenance. (Optional parameter). The databases names in the list, can include wildcard characters from the "Select" instruction "like" clause. <table><tr><th>Wildcard character</th><th>Description</th><th>Example</th></tr><tr><td>%</td><td>Any string of zero or more characters.</td><td>'%base%' selects all databases names that contains the word "base" anywhere in the database name.</td></tr><tr><td>_ (underscore)</td><td>Any single character.</td><td>'an_' selects any three-letter database names beginning with the letters "an", like "An1" or "An9" or "AnF".</td></tr><tr><td>[0-9]</td><td>Any single character between 0 and 9.</td><td>'base[5-9]' selects any five-letter database names beginning with the letters "base", and followed by a number between 5 and 9, like « base5 » or « base9 ».</td></tr><tr><td>[]</td><td>The underscore character.</td><td>'%[]test' selects any database names ending with the characters "_test". The underscore between brackets is interpreted as a character instead of a wildcard character. It is the same behaviour with '%', ']' and '['.</td></tr></table>	Wildcard character	Description	Example	%	Any string of zero or more characters.	'%base%' selects all databases names that contains the word "base" anywhere in the database name.	_ (underscore)	Any single character.	'an_' selects any three-letter database names beginning with the letters "an", like "An1" or "An9" or "AnF".	[0-9]	Any single character between 0 and 9.	'base[5-9]' selects any five-letter database names beginning with the letters "base", and followed by a number between 5 and 9, like « base5 » or « base9 ».	[]	The underscore character.	'%[]test' selects any database names ending with the characters "_test". The underscore between brackets is interpreted as a character instead of a wildcard character. It is the same behaviour with '%', ']' and '['.
Wildcard character	Description	Example														
%	Any string of zero or more characters.	'%base%' selects all databases names that contains the word "base" anywhere in the database name.														
_ (underscore)	Any single character.	'an_' selects any three-letter database names beginning with the letters "an", like "An1" or "An9" or "AnF".														
[0-9]	Any single character between 0 and 9.	'base[5-9]' selects any five-letter database names beginning with the letters "base", and followed by a number between 5 and 9, like « base5 » or « base9 ».														
[]	The underscore character.	'%[]test' selects any database names ending with the characters "_test". The underscore between brackets is interpreted as a character instead of a wildcard character. It is the same behaviour with '%', ']' and '['.														
@ExcDb	One database to exclude, or many using a database list, or many more using a wildcard database list. (Optional parameter). When left empty, no databases are excluded for maintenance. If specified, exclusion processing applies to resulting database list selected by @IncDb processing, which means all databases if @IncDb is an empty string.															
@ExcDbFromPolicy_CheckFullRecoveryModel	Database list to be excluded from the policy check of the full recovery model. (Optional parameter). If the parameter is absent; no database is excluded from the policy check. Maint.YourSqlDbA_DoMaint, will send an error email when a database is in Simple Recovery model and this database is not listed in the @ExcDbFromPolicy_CheckFullRecoveryModel parameter. Each database name filter must be on its own line between the quote pair. The databases name filter list can include wildcard characters as describe in the above @IncDb parameter.															
@EncryptionAlgorithm	This feature is optional, so the parameter. Read important notice about Setup for encrypted backup. Failure to perform recommended setup may lead to failure when attempting to restore backups! nvarchar(10) This feature presently nullify log backups option to the same file. So a new backup file is created for each log backup.															
@EncryptionCertificate	This feature is optional, so the parameter. Read important notice about Setup for encrypted backup. Failure to perform recommended setup may lead to failure when attempting to restore backups! nvarchar(100) This feature presently nullify log backups option to the same file. So a new backup file is created for each log backup.															

Samples:

A. Perform full backups and complete maintenance on all user databases

The following example performs full backups on all user databases. It automatically excludes the system databases.

Exec **YourSQLDbA.Maint.YourSQLDbA_DoMaint**

```

@oper = 'YourSQLDbA_Operator' -- operator used to email report
-- job name displayed in email report
, @MaintJobName = 'YourSQLDbA: DoInteg, DoUpdateStats, DoReorg, Full backups'
, @DoInteg = 1 -- performs database integrity test
, @DoUpdStats = 1 -- performs update statistics
, @DoReorg = 1 -- performs reorganize, rebuild
, @DoBackup = 'F' -- performs full backups and initial log backups

, @FullBackupPath = 'G:\Sql\Backups\' -- full backup path destination
, @LogBackupPath = 'G:\Sql\Backups\' -- log backup path destination

```

```

, @FullBkpRetDays = 0      -- Flush all previous database backups
, @LogBkpRetDays = 7      -- Flush log backups older than (7) days
, @SpreadUpdStatRun = 7   -- Spreads Update Stats over 7 executions

-- Maximum number of consecutive days of failed full backups allowed
-- for a database before putting that database (Offline).
, @ConsecutiveDaysOfFailedbackupsToPutDbOffline= 9999

, @MirrorServer = 'MirrorSQLServ\SQL'

, @IncDb =      -- Each database inclusion filter must be
               -- on its own line between the following quote pair
'
'
, @ExcDb =      -- Each database exclusion filter must be
               -- on its own line between the following quote pair
'
'

, @ExcDbFromPolicy_CheckFullRecoveryModel =      -- Each database exclusion filter must be
                                                -- on its own line between the following quote pair
'
'

```

B. Perform full backups on some user databases

The following example performs full backups on databases which name starts with "Sales" but exclude of this result those which finish with "Temp" plus one or 2 digits.

For example Databases Sales, SalesGeorge, SalesTemp1, SalesTemp25 are first selected by @IncDb parameter, but SalesTemp1 and SalesTemp25 are excluded by @ExcDb. So the final result is that maintenance is going to apply only to Sales, SalesGeorge.

Exec **YourSQLDb.maint>YourSQLDb_DoMaint**

```

    @oper = 'YourSQLDb_operator'      -- operator used to email report
, @MaintJobName = 'YourSQLDb: Sales Full backups' -- job name displayed in email report
, @DoBackup = 'F'                    -- performs full backups and initial log backups
, @FullBackupPath = 'G:\Sql\Backups\' -- full backup path destination
, @LogBackupPath = 'G:\Sql\Backups\'  -- log backup path destination

, @FullBkpRetDays = NULL      -- No cleanup of the full backups
, @LogBkpRetDays = 7         -- Flush log backups older than (7) days

, @IncDb =      -- Each database inclusion filter must be on its own line
               -- between the following quote pair
'
Sales%
'
, @ExcDb =      -- Exclude databases that ends with "temp" followed by
               -- a single digit, or a double digit
'
%temp[0-9]
%temp[0-9][0-9]
'

```

C. Perform only log backups on all user databases

The following example performs only the log backups on all databases.

Exec **YourSQLDb.maint>YourSQLDb_DoMaint**

```

    @oper = 'YourSQLDb_operator'      -- operator used to email report
, @MaintJobName = 'YourSQLDb: Log backups' -- job name displayed in email report
, @DoBackup = 'L'                    -- performs log backups only
, @FullBackupPath = 'G:\Sql\Backups\' -- full backup path destination
, @LogBackupPath = 'G:\Sql\Backups\'  -- log backup path destination
, @IncDb =      -- Each database inclusion filter must be on its own line
               -- between the following quote pair
'
'
, @ExcDb =      -- Each database exclusion filter must be on its own line
               -- between the following quote pair
'
'

```

Installation related procedures

June 26, 2014 5:27 PM

[Install.InitialSetupOfYourSQLDb](#) is the mean by which Sql Agent jobs and email reporting are configured.

[Install.PrintVersionInfo](#) print the current YourSqlDb version.

[Install.UpdateMaintenanceTasksParam](#) can be used as a quick search and replace of YourSqlDb parameters tasks if you customized default install to have many more jobs than the usual two SQL Agent single step jobs.

[Install.AddOrReplaceMaintenance](#) can be used to add/replace a maintenance task with its default parameter. If the @jobNameSuffix parameter, appended to the default YourSqlDb job names makes a match to an existing job name, it is replaced, otherwise it is created. Email reporting stays the same as the one configured by [Install.InitialSetupOfYourSqlDb](#).

Install.InitialSetupOfYourSQLDb

June 20, 2014 4:51 PM

Purpose : Stored procedure to make YourSqlDb tools automatically runnable by adding 2 SQL Server Agent jobs and doing some other setup actions required to achieve email related to job notifications.

- Set up Database Mail to add Database Mail Profile and Database mail Account.
Database mail account name is defined using server and instance name (if not a default instance), and supplied mail server name through the supplied parameter.
- Set SQL Server Agent to use Database Mail Profile.
Set operator "YourSQLDb_Operator" that maps to @email parameter for maintenance email and add it to SQL Agent Operators.
- Setup two SQL Agent Tasks to run **Maint.YourSQLDb_DoMaint** maintenance with default parameters at the exception of few maintenance parameters which can be supplied through this procedure.
For other parameters, they can be changed directly in SQL Agent YourSqlDb jobs. The first job added performs all tasks related to daily maintenance, the second performs log backups through the day.
Both jobs have their own schedules that can be changed independently.

Sample :

```
Exec YourSQLDb.Install.InitialSetupOfYourSQLDb
    @email = 'myAdmin@myDomain.com' -- Email recipients (or distribution list)
, @SmtpMailServer = 'myMailServer' -- Mail server that accept SMTP mail
, @ConsecutiveDaysOfFailedBackupsToPutDbOffline = 3 -- for part time DBAs...
```

OR if you want to specify a different location for backups than the default one specified in server properties / database settings

```
Exec YourSQLDb.Install.InitialSetupOfYourSQLDb
    @FullBackupPath = 'D:\Sql\Backups' -- full backup path destination
, @LogBackupPath = 'D:\Sql\Backups' -- log backup path destination
, @email = 'myAdmin@myDomain.com' -- Email recipients (or distribution list)
, @SmtpMailServer = 'myMailServer' -- Mail server that accept SMTP mail

, @ConsecutiveDaysOfFailedBackupsToPutDbOffline = 9999
-- Maximum number of consecutive days of failed full backups
allowed
-- for a database before putting that database (Offline).
```

Parameter name	parameter description
@FullBackupPath	Optional. Location where full database backup file (.Bak) are going to be stored. UNC are allowed. (nvarchar(512)). Default : value specified in server properties\database settings
@LogBackupPath	Optional. Location where log database backup file (.Trn) are going to be stored. UNC are allowed. (nvarchar(512)). Default : value specified in server properties\database settings
@email	Email recipients (or distribution list). Each item must be separated by semicolon; Limited to 100 characters despite wider parameters, because SQL Server Agent limits operator definition to 100 characters. (nvarchar(512))
@SmtpMailServer	Mail server that will be authorized to accept SMTP mail from your SQL Server machine. By default authentication is set to anonymous, but it can be modified later in database mail account generated as an account of the generated YourSQLDb_MailProfile. (nvarchar(128))
, @SmtpMailPort int = 25	Optional: When omitted Smtp port is the official port, 25
, @SmtpMailEnableSSL bit = 0	Optional: When omitted SMTP do not use SSL
@ConsecutiveDaysOfFailedbackupsToPutDbOffline	Maximum number of consecutive days of failed full backups allowed for a database before putting that database (Offline). One successful full backup will reset to zero the counter of consecutive days of failed full backups for that database. Last resort action to inform database administrators

Normally YourSQLDb send's an email to the SQL administrators when there is a full backup failure. However if these emails stop reaching their recipients, there is a need for a last resort mechanism to inform the administrators that the data is not protected.

The maintenance parameter **@ConsecutiveDaysOfFailedBackupsToPutDbOffline** instructs YourSQLDb to put offline a database as a last resort to signal that full backups of a given database have failed for the specified number of consecutives days.

When the full backup maintenance job is executed a counter is incremented once a day for each database for which the full backup failed.

If this days counter is equal to the maximum number of consecutive failed full backup days allowed by the parameter **@ConsecutiveDaysOfFailedBackupsToPutDbOffline** , the corresponding database is put offline.

The counter is reset to zero when the full backup succeed's for the database.

This feature is a **last resort** to attract attention of a database administrator or application users to a potentially dangerous backup problem that can lead to data loss.

Note that the suspension of service can occur only at the end of the full backup maintenance job which is usually in the middle of the night.

Important: Putting a database offline implies a temporary suspension of service. You may not want that to happen. Even if no backups are available for the database.

You must set a value for this number of days allowed for full backup failures.

The number of days to choose depends on the availability level of the database administrators and of the risk tolerance for data loss.

When full backups don't work, it's usually because the disk is almost full. Most of the time the log backups continues even when the full backups didn't. This is because log backups need less disk space than full backups.

The log backups allows the restoration of a database from the last available full backup.

Keep in mind that the greater the number of days gone without full backups or even without transaction logs backups, the riskier it is to lose data in case of a SQL server failure.

You have to decide the threshold number of days of full backup failures that will cause the temporary suspension of service for the database.

Here are 3 scenarios to choose from

1. If you think that your organisation has fail safe SQL management practices to guarantee that backups are checked every day, you can turn off this behaviour by using a value of 9999 days:

@ConsecutiveDaysOfFailedbackupsToPutDbOffline = 9999

Specifying a great number of days for the **@ConsecutiveDaysOfFailedbackupsToPutDbOffline**parameter will effectively disable it.

2. If you think that there would not be a database administrator available every morning to verify the production of the full backups or to put the databases online in case of a full backup failure, you can use a value of 20 days :

@ConsecutiveDaysOfFailedbackupsToPutDbOffline= 20

3. If you think that there will always be a database administrator available every morning to verify the production of the full backups or to put the databases online in case of a full backup failure, then you can set a value of 4 days by running the following command in SQL Management Studio:

@ConsecutiveDaysOfFailedbackupsToPutDbOffline= 4

@FullBkExt	Optional : By default full backup have ".bak" extension (nvarchar(7))
@LogBkExt	Optional : By default log backup have ".trn" extension (nvarchar(7))
@sourceEmail	Optional : Change the sender name to the sender of your choice. (nvarchar(512))
@emailServerAccount	Optional: If basic authentication is required, email server account can be specified. (nvarchar(512))
@emailServerPassword	Optional: If basic authentication is required, email server account password can be specified. (nvarchar(512))
@FullMaintenanceScript	Optional: Output parameter that helps to receive the generated script of the full maintenance job unique step. Can be used for a Setup for a SQL Express instance in which there is no SQL Agent, so it can be scheduled by other means.
@LogBackupScript	Optional: same idea as @fullMaintenanceScript, but for the log backup maintenance script.

Install.PrintVersionInfo

June 20, 2014 5:01 PM

Purpose: Prints the YourSQLDb version.

Sample

```
Exec YourSQLDb.Install.PrintVersionInfo
```

Install.AddOrReplaceMaintenance

March 1, 2016 11:54 AM

Purpose :

- Recreates or add a new set of full/log maintenance.
- Setup two SQL Agent Tasks to runs **Maint.YourSQLDbA_DoMaint** maintenance with default parameters, at the exception of backup path parameters which are supplied through this procedure.

Sample :

Exec **YourSQLDbA.Install.AddOrReplaceMaintenance**

```
@JobNameSuffix = ' JobForNeedXXX'      -- full backup path
, @FullBackupPath = 'G:\Sql\Backups'    -- full backup path destination
, @LogBackupPath = 'G:\Sql\Backups'     -- log backup path destination
, @ConsecutiveDaysOfFailedbackupsToPutDbOffline = 9999
      -- Maximum number of consecutive days of failed full backups allowed
      -- for a database before putting that database (Offline).
```

Parameter name	parameter description
@JobNameSuffix	Add this suffix to default job names, YourSQLDbA_FullBackups_And_Maintenance and YourSQLDbA_LogBackups. If a name of a job exists with the same suffix, replace the job, otherwise creates it.
@FullBackupPath	Location where full database backup file (.Bak) are going to be stored. UNC are allowed. (nvarchar(512))
@LogBackupPath	Location where log database backup file (.Trn) are going to be stored. UNC are allowed. (nvarchar(512))
@ConsecutiveDaysOfFailedbackupsToPutDbOffline	Maximum number of consecutive days of failed full backups allowed for a database before putting that database (Offline). One successful full backup will reset to zero the counter of consecutive days of failed full backups for that database. Last resort action to inform database administrators Normally YourSQLDbA send's an email to the SQL administrators when there is a full backup failure. However if these emails stop reaching their recipients, there is a need for a last resort mechanism to inform the administrators that the data is not protected. The maintenance parameter @ConsecutiveDaysOfFailedBackupsToPutDbOffline instructs YourSQLDbA to put offline a database as a last resort to signal that full backups of a given database have failed for the specified number of consecutives days. When the full backup maintenance job is executed a counter is incremented once a day for each database for which the full backup failed. If this days counter is equal to the maximum number of consecutive failed full backup days allowed by the parameter @ConsecutiveDaysOfFailedBackupsToPutDbOffline , the corresponding database is put offline. The counter is reset to zero when the full backup succeed's for the database. This feature is a last resort to attract attention of a database administrator or application users to a potentially dangerous backup problem that can lead to data loss. Note that the suspension of service can occur only at the end of the full backup maintenance job which is usually in the middle of the night. Important: Putting a database offline implies a temporary suspension of service. You may not want that to happen. Even if no backups are available for the database. You must set a value for this number of days allowed for full backup failures. The number of days to choose depends on the availability level of the database administrators and of the risk tolerance for data loss. When full backups don't work, it's usually because the disk is almost full. Most of the time the log backups continues even when the full backups didn't. This is because log backups need less disk space than full backups. The log backups allows the restoration of a database from the last available full backup. Keep in mind that the greater the number of days gone without full backups or even without transaction logs backups, the riskier it is to lose data in case of a SQL server failure. You have to decide the threshold number of days of full backup failures that will cause the temporary suspension of service for the database. Here are 3 scenarios to choose from

1. If you think that your organisation has fail safe SQL management practices to guarantee that backups are checked every day, you can turn off this behaviour by using a value of 9999 days:

`@ConsecutiveDaysOfFailedbackupsToPutDbOffline = 9999`

Specifying a great number of days for the `@ConsecutiveDaysOfFailedbackupsToPutDbOffline` parameter will effectively disable it.

2. If you think that there would not be a database administrator available every morning to verify the production of the full backups or to put the databases online in case of a full backup failure, you can use a value of 20 days :

`@ConsecutiveDaysOfFailedbackupsToPutDbOffline= 20`

3. If you think that there will always be a database administrator available every morning to verify the production of the full backups or to put the databases online in case of a full backup failure, then you can set a value of 4 days by running the following command in SQL Management Studio:

`@ConsecutiveDaysOfFailedbackupsToPutDbOffline= 4`

Install.UpdateMaintenanceTasksParam

June 20, 2014 5:03 PM

Purpose :

- Replaces the value of a parameter in all SQL Server Agent jobs which executes the **Maint.YourSQLDbA_DoMaint** procedure. Intended for people who have many many YourSqlDbA tasks. Usually it is easier to go directly on the single task of one of the two SQL Server Agent job.

Parameter name	Parameter description
@paramName	Sysname Parameter name for which the value must be changed.
@paramValue	Nvarchar(max) New value for the parameter.

Samples:

A. Replacing the value for the @ConsecutiveFailedbackupsDaysToPutDbOffline parameter

```
Exec YOURSQLDBA.Install.UpdateMaintenanceTasksParam
-- Parameter name for which the value must be changed.
@paramName = '@ConsecutiveFailedbackupsDaysToPutDbOffline'
-- New value for the parameter.
, @paramValue = '20'
```

B. Replacing the value for the @LogBkpRetDays parameter

```
Exec YOURSQLDBA.Install.UpdateMaintenanceTasksParam
@paramName = '@LogBkpRetDays' -- Parameter name for which the value must be changed.
, @paramValue = '14' -- New value for the parameter.
```

Database recovery with YourSqlDBa

June 27, 2014 11:16 AM

Database offline or in emergency mode.

There are two causes possible to that and what to do.

1. Sql Server put the database offline by itself .
 - a. Finding database in offline mode is obviously a serious matter. A offline database can be in this state because one of the drive onto which one of it file it is stored is not accessible.
 - b. Finding a database in emergency mode is a more serious thing. The database internal structures have few or too much inconsistencies that commend to stop them using it now, a do a recovery before data loss could become more severe.
2. YourSqlDBa put the database offline because one of the following reasons.
 - a. When YourSqlDBa puts a database in emergency mode it does it for the same reason that Sql Server would have done it. It just detects faster integrity problems which increases a lot your chances of recovery of theses database and their data. YourSqlDBa detects internal inconsistencies by the use of the DBCC CheckDb T-SQL command. This database can be recovered easily by using a full database backup of previous day onto which maintenance succeed without putting the database in emergency mode. Then restore the database using this backup file and all log backups files in between.
 - b. When a database is put offline by YourSqlDBa, this is because you specified at initial setup of YourSqlDBa to do it so if no successful backups of that database where done after the specified number of days specified. Did you notice that you were no longer receiving YourSqlDBa email alerts or are you just ignoring them? Don't panic! Bring them online by right clicking on database to get contextual menu and find the option to bring it online. A faster way to do it if you have many is to run [Maint.BringBackOnlineAllBdInEmergencyOrOffline](#). Next find out why it happened by displaying backup errors with [Maint.ShowHistoryErrors](#). Solve the problem (most often a lack of backup space, or an inaccessible or lack of privileges to backup destination), and once ok performs [Maint.SaveDbOnNewFileSet](#). Last find out why email messages alert from YourSqlDBa doesn't reach you. Check anti-spam, diagnose database mail with [Maint.DiagDbMail](#).

Database restore

YourSqlDBa does adequate backups and save log backup history by saving regularly MSDB , every time it performs log backups.

YourSqlDBa use a much better naming pattern for backup files and log history backup details to know which files where used and when every backup was performed.

SQL Server management Studio display this history when you want to do a restore. All parameters are there by default to perform a complete restore. This is very easy to select which backup, and it appropriates log backups and do a point-in-time recovery for a given database.

If you loose all databases your need to recover first MASTER. Get some help if you have no clue to do that.

Rerun YourSqlDBa script to re-create YoursqlDBa database.

Once done we suggest to restore MSDB database which contains backup history. I suggest to stop SQL Agent before because it is likely to open a connection to MSDB which prevents restoring it easily.

You can use [Maint.RestoreDb](#) to restore it, if you are not familiar with SQL Management Studio to restore it without backup history. However if you look carefully to SQL Management Studio restore dialog, you'll see on the main screen how to select a backup file. Choose whatever method with which you are comfortable.

From there it starts to be easier. You get backup history for every database. From there it is really easy to restore a database, all is there by default, just click ok.

Maint.BringBackOnlineAllBdInEmergencyOrOffline

June 25, 2014 5:55 PM

Purpose : Bring back online quickly all databases that are offline or in emergency mode.

Usage notes:

YourSQLDb may put a database in emergency mode when an integrity error is detected.

YourSQLDb may put a database offline as a last resort to signal that full backups of that database have failed for the specified number of consecutive days.

When the source of the problem is resolved, the SQL administrators can bring back online all databases by executing the **Maint.BringBackOnlineAllBdInEmergencyOrOffline** procedure.

Sample :

```
Exec YourSQLDb.Maint.BringBackOnlineAllBdInEmergencyOrOffline
```

Maint.RestoreDb

June 25, 2014 6:07 PM

Purpose : Restore database from a backup copy made by **Maint.SaveDbCopyOnly**.

Usage notes: @ReplaceExistingDb parameter is equivalent of With replace option of T-SQL Restore command. Contrary to the Restore command, which accept restoring a database from one of its backups, even if the replace option is missing, here @ReplaceExistingDb set to 0 forbids overwriting an existing database. To backup user must be member of db_backupOperator or db_owner, and to restore he must be member of db_creator server role.

Parameter name	parameter description
@TargetDb	Name of the destination database to create. It must not already exist. (sysname)
@PathAndFilename	Full path and file name of the backup file (NVarchar(512))
@ReplaceExistingDb	Set to 1 to overwrite existing database. When set to 0 and database exists, no restore is performed. (Int)

Sample :

```
Exec YourSQLDb.Maint.RestoreDb
    @TargetDb = 'SalesDb' -- name of the database
-- path and file name of the backup
    , @PathAndFilename = 'g:\sql\backups\SalesDbBeforeMonthlyReports.bak'
-- specify to restore over an existing database
-- or a database for which SQL wants you to save log tail before restoring,
-- if you don't care about existing unsaved log transactions
    , @ReplaceExistingDb = 0
```

Delegation for backup/restore procedures and database upgrades

June 27, 2014 10:36 AM

Delegated database management

YourSqlDb provides a controlled delegation mechanism for database operations that normally require elevated SQL Server permissions.

Why delegation is needed

Application owners and senior support users sometimes need to perform limited database administration tasks without becoming SQL Server administrators.

A common scenario is refreshing a non-production database from production before testing an application upgrade. The application owner may need to:

- create a backup of an authorized source database;
- restore that database under a non-production name;
- prepare a database for an application upgrade;
- test the upgraded database;
- roll back a failed upgrade;
- remove backup files created by these delegated operations.

Granting sysadmin privileges for these tasks would provide far more authority than necessary. Delegation limits the user to explicitly authorized databases and enforces additional safeguards around restore targets.

Security model

The delegated procedures execute with the permissions required to perform backup, restore, rename, and recovery operations. Before doing any privileged work, they validate the caller using:

ORIGINAL_LOGIN()

Non-sysadmin users must have a row in:
YourSqlDb.Maint.DelegatedDbManagement

The validation checks:

- the caller's original login;
- whether the source database is authorized;
- whether the requested operation is permitted;
- whether a restore target follows the required naming convention.

A delegated user cannot restore over the source database. For a source database named Payroll, a restore target must begin with:

Payroll_

Examples:

Payroll_Joe

Payroll_UpgradeTest

Payroll_2026_Validation

The following names are not permitted:

Payroll

Payroll2026

PayRollTest

Important: Names beginning with Payroll_ form a delegated namespace. Production databases should not use names within that namespace because a delegated restore

could replace an existing authorized target.

Delegation controls database scope. It does not replace appropriate security on the backup directories accessible to the SQL Server service account.

Delegated backup, restore, and cleanup operations

The following procedures support delegated execution:

- **Maint.SaveDbOnNewFileSet**
Starts a new backup file set using YourSqlDb's backup conventions and maintenance configuration.
- **Maint.SaveDbCopyOnly**
Creates a copy-only backup at a specified path without affecting the normal backup chain.
- **Maint.DuplicateDb**
Creates a copy-only backup of the source database and restores it under an authorized target name.
- **Maint.DuplicateDbFromBackupHistory**
Duplicates a database using backup information recorded in SQL Server backup history, requiring fewer path parameters than **Maint.DuplicateDb**.
- **Maint.RestoreDb**
Restores a backup under an authorized target database name.
- **Maint.DeleteOldBackups**
Removes eligible backup files associated with delegated database variants. A non-sysadmin user cannot use this procedure to clean backups belonging to the source database itself or to unrelated databases.

Delegating application upgrades

YourSqlDb also provides a controlled workflow for application upgrades.

Application upgrades may require exclusive access to the database. Obtaining reliable exclusive access can be difficult when services, scheduled tasks, monitoring tools, or user sessions reconnect automatically.

The maintenance-mode workflow solves this by:

1. terminating existing connections;
2. renaming the database from, for example, Payroll to Payroll_MaintenanceMode;
3. creating a recovery point before the upgrade;
4. allowing the application owner to upgrade and test the renamed database;
5. restoring the pre-upgrade state if necessary;
6. returning the database to its original name.

Because applications normally connect using the original database name, they can no longer reconnect after the database is renamed.

Maintenance-mode procedures

- **Maint.PrepareDbForMaintenanceMode**
Terminates current connections and renames the database using the fixed `_MaintenanceMode` suffix. It records the original recovery model and creates a marked transaction that can be used as a restore target.
When backup history and a transaction log backup location are available, it takes a log backup. A separate copy-only backup can also be created by specifying `@PathOfBackupBeforeMaintenance`.

- **Maint.RestoreDbAtStartOfMaintenanceMode**
Restores the renamed database to its state at the beginning of maintenance. The database remains under its `_MaintenanceMode` name, allowing the upgrade to be attempted again.
- **Maint.ReturnDbToNormalUseFromMaintenanceMode**
Restores the original recovery model, renames the database to its original name, and makes it available to applications again.

After a successful upgrade:

PrepDbForMaintenanceMode

- perform and validate the upgrade
- **ReturnDbToNormalUseFromMaintenanceMode**

After a failed upgrade:

PrepDbForMaintenanceMode

- upgrade fails
- **RestoreDbAtStartOfMaintenanceMode**
- retry or abandon the upgrade
- **ReturnDbToNormalUseFromMaintenanceMode**

Maint.ReturnDbToNormalUseFromMaintenanceMode must be executed when maintenance ends, whether the upgrade succeeded or was rolled back.

Granting delegated access

A sysadmin grants access by inserting one row per login into:

Maint.DelegatedDbManagement

The table contains these columns:

Column	Purpose
LoginName	Login returned by ORIGINAL_LOGIN()
SourceDatabaseList	Databases authorized for general delegated backup, restore, duplication, cleanup and maintenance-mode operations
MaintenanceModeDatabaseList	Additional databases authorized only for the maintenance-mode workflow
CreatedAt	Date and time when the row was created
CreatedBy	Login that created the row

Use the exact login name returned in the delegated user's session:

```
SELECT ORIGINAL_LOGIN();
```

A sysadmin can then grant access:

```
USE YourSqlDb;
GO
INSERT INTO Maint.DelegatedDbManagement
(
    LoginName,
    SourceDatabaseList,
    MaintenanceModeDatabaseList
)
VALUES
(
    N'DOMAIN\Joe',
    N'Payroll,Testfct',
```

```
N'Accounting'  
);
```

In this example:

- Joe may perform delegated backup, restore and duplication operations using Payroll and Testfct as source databases.
- Joe may use the application-upgrade workflow on Payroll, Testfct, and Accounting.
- A database listed in SourceDatabaseList does not need to be repeated in MaintenanceModeDatabaseList.

To modify the delegation:

```
UPDATE Maint.DelegatedDbManagement  
SET  
    SourceDatabaseList = N'Payroll,Testfct',  
    MaintenanceModeDatabaseList = N'Accounting,Inventory'  
WHERE LoginName = N'DOMAIN\Joe';
```

To revoke all delegated access:

```
DELETE FROM Maint.DelegatedDbManagement  
WHERE LoginName = N'DOMAIN\Joe';
```

The delegated login should not be given permission to modify this table.

Maint.SaveDbOnNewFileSet

June 25, 2014 6:08 PM

Maint.SaveDbOnNewFileSet

Purpose: Start backups for a given database with a new file set (the pair full backup file and log backup files for a database).

This new file set is going to be the reference set for log backups, which will continue to append log backups to the same file until the next **YourSQLDb.Maint>YourSQLDb_DoMaint** run. In that run, the log backup path location will reset to YourSQLDb_DoMaint backup path locations.

Parameter name	parameter description
@DbName	Database name to backup. (NVarchar(128))
@FullBackupPath	Location where the full database backup file (.Bak) will be stored. UNC are allowed. (NVarchar(512))
@LogBackupPath	Location where the log database backup file (.Trn) will be stored. UNC are allowed. (NVarchar(512))
@oper	Operator name, by default YourSQLDb_Operator. Any user defined SQL Server Agent Operator is allowed. (NVarchar(128))
@MirrorServer	Name of a mirror SQL instance destination to automatically restore the backup. Optional. The Mirroring.AddServer procedure must be executed to record this mirror SQL instance, prior to using it here in the @MirrorServer parameter. Example: 'MirrorSQLServ\SQL' All the SQL logins will be copied from the source instance to the mirror SQL instance. NULL = use the last mirror SQL instance used by the global maintenance. (sysname)
@EncryptionAlgorithm	Read important notice about Setup for encrypted backup. Failure to perform recommended setup may lead to non-restorable backups! Under version 6.5.9.3 this parameter must be set to empty string. nvarchar(10)
@EncryptionCertificate	Read important notice about Setup for encrypted backup. Failure to perform recommended setup may lead to non-restorable backups! Under version 6.5.9.3 this parameter must be set to empty string. nvarchar(100)

Sample:

```
Exec YourSQLDb.Maint.SaveDbOnNewFileSet
    @DbName = 'DATABASE' -- Database name to backup.
-- Location where the full database backup file (.Bak) will be stored.
, @FullBackupPath = 'G:\Sql2005Backups\'
-- Location where log database backup file (.Trn) will be stored.
, @LogBackupPath = 'G:\Sql2005Backups\'
-- Operator name, by default YourSQLDb_Operator.
, @oper = 'YourSQLDb_Operator'
, @MirrorServer='SqlM\Sales'
```

Maint.SaveDbCopyOnly

June 25, 2014 6:08 PM

Purpose : Make a backup copy without disturbing log backups performed with regular log backup maintenance. Use the special `With copy_only` parameter, to avoid disturbing backup log sequence and database LSN number. These types of backups are not recorded to the backup log history and can't be restored from SQL Management Studio graphical interface for database restore. The must be restore with the supplied **Maint.RestoreDb** procedure.

Usage notes: The procedure checks if the call has at least rights to backup the database. Being member of `db_backupOperator` fix database role is the minimal rights set required. It is convenient because it doesn't allow end user other access to the database (unless some rights are added to him through public database role). This limit allow to put credentials into a script that calls `Sqlcmd` without the risk of exposing database content or giving more access to this user. Being constrained to put a database backup to a designed location accessible only from the database engine avoid disclosure of information.

Parameter name	parameter description
@DbName	Database name to backup (sysname)
@PathAndFilename	Full path and file name of the backup file.(NVarchar(512))
@errorN	Output error code for batches (int)

Sample :

```
Exec YourSQLDb.Maint.SaveDbCopyOnly
    @DbName = 'SalesDb'           -- name of the database
, @PathAndFilename = 'g:\sql\backups\SalesDbBeforeMonthlyReports.bak'
```

Maint.DuplicateDb, Maint.DuplicateDbFromBackupHistory

June 25, 2014 6:05 PM

Purpose : Creates a duplicate of an existing database. Target Db must not exists.

There is a big only difference between both procedures. **YourSQLDb.Maint.DuplicateDbFromBackupHistory** do not have a to do a backup. It finds itself the latest full and following log from backup history. For the same reason, it doesn't need @pathAndFileName parameter.

Parameter name	parameter description
@sourceDb	Name of the source database to duplicate (sysname)
@TargetDb	Name of the destination database to create. It must not already exists. (sysname)
@PathAndFilename	Full path and file name of the backup file. (NVarchar(512))
@KeepBackupFile	When set to 0, backup file is removed after restore. (bit)

Sample :

```
Exec YourSQLDb.Maint.DuplicateDb
    @sourceDb = 'Sales' -- Source database
, @TargetDb = 'SalesBeforeMonthlyReport' -- Target database
, @PathAndFilename = 'g:\sql\backups\SalesBeforeMonthlyReports.bak' -- .bak
, @KeepBackupFile = 0 -- don't keep backup file once restore is done
```

Maint.DeleteOldBackups

June 25, 2014 5:56 PM

Purpose : Delete old backup files.

This procedure is usually executed from a global maintenance job in SQL Server Agent, but it can be executed alone in its own job in SQL Server Agent, or in a SQL Server Management Studio query window.

Parameter name	parameter description												
@oper	SQL Server Agent Operator. Any valid operator name is allowed. By default 'YourSQLDbOperator'. (NVarChar(200))												
@MaintJobName	SQL Server Agent job name logged into the maintenance history. Any valid job name is allowed. By default 'YourSQLDb: DoInteg,DoUpdateStats,DoReorg,Full backups'. (NVarChar(200))												
@Path	Full path of the backup directory. Mandatory. A UNC path is allowed. Example A : 'G:\Sql\Backups' Example B : '\\SQL\Backups' (NVarChar(512))												
@BkpRetDays	Number of days to retain the backup files selected by their corresponding database name. Backup files that are too old, will be deleted from the backup directory. NULL = no cleanup (default value). Int												
@BkpRetDaysForUnselectedDb	Number of days to retain the backup files <u>not</u> selected by their corresponding database name. Backup files that are too old, will be deleted from the backup directory. This parameter permits the deletion of backup files for databases that don't exist anymore in the SQL server. That could happen for a test database, taken in backup, deleted after a test, and no more selected by @incDb or @excDb parameters. NULL = no cleanup (default value). Int												
@RefDate	Reference date for the cleanup of backup files. This date can be provided in the following form '2009-05-14 17:15' NULL = use the current date and time. (default value). Datetime												
@extension	File extension for the backup files to be deleted. The file extension can be of any length. Examples : '.bak' for full backup files, 'trn' for transaction log backup files, " (an empty character string between two single quotes) for any kind of backup file. Default value = ". (Sysname)												
@IncDb	Database filter list to include in the cleanup process. Each database inclusion filter must be on its own line between the quote pair. Each database inclusion filter can include regular characters and wildcard characters used in the SQL « like » operator in the « Select » statement. <table><tr><th>Wildcard character</th><th>Meaning</th><th>Example</th></tr><tr><td>%</td><td>Any string of zero or more characters.</td><td>'%math%' finds all database names with the word 'math' anywhere in the database name.</td></tr><tr><td>_ (underscore)</td><td>Any single character.</td><td>'an_' finds all three- character database names with « an » as the first two characters, like « An1 » or « An9 » or « AnF ».</td></tr><tr><td>[]</td><td>Any single character within the specified range ([a-f]) or set ([abcdef]).</td><td>'base[5-9]' finds all five-character database names with « base » as the first four characters, followed by a number between 5 and 9, like « base5 » or « base9 ».</td></tr></table> @IncDb is an optional parameter. If it is absent, all databases names visible in the SQL server Management Studio database list, are selected.	Wildcard character	Meaning	Example	%	Any string of zero or more characters.	'%math%' finds all database names with the word 'math' anywhere in the database name.	_ (underscore)	Any single character.	'an_' finds all three- character database names with « an » as the first two characters, like « An1 » or « An9 » or « AnF ».	[]	Any single character within the specified range ([a-f]) or set ([abcdef]).	'base[5-9]' finds all five-character database names with « base » as the first four characters, followed by a number between 5 and 9, like « base5 » or « base9 ».
Wildcard character	Meaning	Example											
%	Any string of zero or more characters.	'%math%' finds all database names with the word 'math' anywhere in the database name.											
_ (underscore)	Any single character.	'an_' finds all three- character database names with « an » as the first two characters, like « An1 » or « An9 » or « AnF ».											
[]	Any single character within the specified range ([a-f]) or set ([abcdef]).	'base[5-9]' finds all five-character database names with « base » as the first four characters, followed by a number between 5 and 9, like « base5 » or « base9 ».											

	(NVarchar(max))
@ExcDb	Database filter list to exclude exclude from the list of database names selected by the @incDb parameter. Each database exclusion filter must be on its own line between the quote pair. Each database exclusion filter can include regular characters and wildcard characters used in the SQL « like » operator in the « Select » statement. @ExcDb is an optional parameter. If it is absent, no databases names selected by the @IncDb are excluded. (NVarchar(max))
@JobNo	Current job number to record maintenance messages in the history table of YourSQLDb. This parameter is generated by the global maintenance and can be used here to add cleanup messages to the global maintenance messages, in the same job number. NULL = creates a new entry in the maintenance history table of YourSQLDb. (default value). Int

Samples :

A. Delete all previous backup files

This sample delete's all backup files for all databases.

Let's suppose that the server execute a copy of your backup files on another server. This could be done with the XCOPY command.

The following sample could delete the backup files, after their copy on tape.

Exec YourSQLDb.Maint.DeleteOldBackups

```
@path = 'G:\Sql\Backups\ForXCOPY\' -- Full path of the backup directory.

, @BkpRetDays = 0 -- Delete backup files selected by their file name
                  -- in the selected database list generated by
                  -- the @IncDb and @ExcDb parameters,
                  -- and for these files
                  -- their age in minutes
                  -- must be greater than (@BkpRetDays * 1440) minutes.

, @BkpRetDaysForUnselectedDb = 0

                  -- Delete backup files not selected by their file name
                  -- in the selected database list generated by
                  -- the @IncDb and @ExcDb parameters
                  -- and for these files
                  -- their age in minutes
                  -- must be greater than (@BkpRetDaysForUnselectedDb * 1440) minutes.
                  -- This parameter permits the deletion of backup files for
                  -- databases that don't exist anymore in the SQL server.

, @extension = '' -- Select backup files with any extension (.bak .trn and others).
```

B. Delete all backup files, for some selected databases

This sample delete's all backup files for databases names ending with '_test', but retains files '_test' containing the word 'Archive'.

For example Sales_test, SalesGeorge_test are selected by @IncDb, but SalesGeorge_archive_Test is excluded by @ExcDb.

Exec YourSQLDb.Maint.DeleteOldBackups

```
@path = 'G:\Sql\Backups\ForXCOPY\' -- Full path of the backup directory.

, @BkpRetDays = 0 -- Delete backup files selected by their file name
                  -- in the selected database list generated by
                  -- the @IncDb and @ExcDb parameters,
                  -- and for these files,
                  -- their age in minutes
                  -- must be greater than (@BkpRetDays * 1440) minutes.

, @BkpRetDaysForUnselectedDb = NULL
                  -- Ignore backup files not selected by their file name
```

```

-- in the selected database list generated by
-- the @IncDb and @ExcDb parameters.

, @extension = '' -- Select backup files with any extension (.bak .trn and others).

, @IncDb = -- Each database inclusion filter must be
-- on its own line between the following quote pair
'
%[_]test
'
, @ExcDb = -- Each database exclusion filter must be
-- on its own line between the following quote pair
'
%Archive%
'

```

C. Execute the DeleteOldBackups procedure in a SQL Server Management Studio query window, to delete all backup files, for some selected databases

This sample delete's all backup files for databases names ending with '_test', but retains files '_test' containing the word 'Archive'.

For example Sales_test, SalesGeorge_test are selected by @IncDb, but SalesGeorge_archive_Test is excluded by @ExcDb.

```

Exec YourSQLDBa.Maint.DeleteOldBackups @path = 'G:\Sql\Backups\ForXCOPY\' -- Full path of
the backup directory.

```

```

, @BkpRetDays = 0 -- Delete backup files selected by their file name
-- in the selected database list generated by
-- the @IncDb and @ExcDb parameters,
-- and for these files,
-- their age in minutes
-- must be greater than (@BkpRetDays * 1440) minutes.

, @BkpRetDaysForUnselectedDb = NULL

-- Ignore backup files not selected by their file name
-- in the selected database list generated by
-- the @IncDb and @ExcDb parameters.

, @extension = '' -- Select backup files with any extension (.bak .trn and others).

, @IncDb = -- Each database inclusion filter must be
-- on its own line between the following quote pair ' %[_]test '

, @ExcDb = -- Each database exclusion filter must be
-- on its own line between the following quote pair ' %Archive% '

```

Exclusive access and backups for database maintenance

June 27, 2014 11:20 AM

Database maintenance are an important part of a DBA job. By maintenance I mean an application software upgrade that requires a database maintenance (some schema modification for example).

Usually application software maker is going to require you to ensure that your database is not accessible by anybody, and to have the most current backup before maintenance, so you are going to be able to restore it in case the maintenance goes wrong.

In today interconnected world having a complete exclusive access to a database is not an easy task. Putting it in single_user mode or in restricted user access is often not a adequate solution. A software service other than your application may try to connect to your database, locking it from any other access. The only safe way to warrant that the database is dedicated to you is to do quickly in sequence: put it offline, put it back offline, and rename it. By renaming it, software services, especially those who makes connection pooling won't find anymore the database, so won't reconnect to it. The same hols true for actual user of any application.

YourSqlDba supply three clever store procedures for that specific purpose.

[Maint.PrepareDbForMaintenanceMode](#) get a most up to date backup/snapshot of the database.

Putting it strait, if you need to restore because maintenance goes into trouble,

[Maint.RestoreDbAtStartOfMaintenanceMode](#) will use them to allow you to get your database as it was before the maintenance.

If you use SQL Server Enterprise Edition, [Maint.PrepareDbForMaintenanceMode](#) does a snapshot of the database, which is a very quick way to get a backup equivalent of a database, and a very quick mean to rollback changes to a database. For other editions, if your database is in full recovery mode, it performs a log backup (with log mark for precise point-in-time restore) which is quick and sufficient to do a complete up to date restore of your database, given that daily full backup is there and all other log backups. If it is in simple recovery mode, it performs a full back which is the only mean YourSqlDba supplies to warrant an backup up to date to restore.

In case of maintenance failure that request a database restore you need to bring back database at how it was at start of maintenance. For that purpose you do a

[Maint.RestoreDbAtStartOfMaintenanceMode](#) which brings back the renamed database to its initial state. If your maintenance failed because a problem that can be solved, e.g. like a lack of space, you can then attempt again the maintenance. Either you succeed or you just restore the database with not further attempt to maintenance it, you must return DB to normal use, which just mean to rename it back to its initial name. For that purpose just run

[Maint.ReturnDbToNormalUseFromMaintenanceMode](#). Essentially it renames the database in its current state by removing the same suffix you specified in parameter. This one must be the same as the one you specified when using [Maint.PrepareDbForMaintenanceMode](#).

Maint.PrepareDbForMaintenanceMode

June 25, 2014 6:06 PM

Purpose : This procedure offers a mean to get exclusive access to a database, with any number of connections, by changing the database name. Since the new database name is not known to applications, you are alone with it.

Usage notes :

Some maintenance processes required that a user (usually an application administrator or a DBA) has an exclusive access to the database. External connections from applications or services or users must be kicked out of the database, and prevented from re-accessing the database for the duration of the maintenance process. Ongoing transactions must be rolled back in the disconnection process. Only a database renaming process insures this state. The administrator must have a way to switch quickly to the new database name by having some connections definitions (ODBC data source or connect string configuration files) ready for the new database name and only accessible for him. External connections cannot have access to the database anymore because they still point to the old database name.

This procedure offer a mean to get exclusive access by performing the following actions :

1. Kill all connections to the databases selected with rollback of the transactions.
2. Rename the databases.
3. Nobody is able to connect to the original database since its name is changed.

The new name is build with the original name + '_' + @DbNameSuffixForMaintenance.

1. Put the newly renamed databases in « MULTI_USER ».
2. Do a full backup when the parameter @PathOfBackupBeforeMaintenance is completed.

The backup file name is build with the original database name + '_' + @DbNameSuffixForMaintenance+ '.BAK'.

Parameter name	parameter description
@DbList	One or many databases name, see sample below to see how to specify multiple databases. (NVarchar(max))
@DbNameSuffixForMaintenance	Suffix to add to the database name to rename it. (NVarchar(128))
@PathOfBackupBeforeMaintenance	Path of the backup that will be used to return the database to its current state at this procedure completion, in case of maintenance failure. Recommended, otherwise specify an empty string. When the @PathOfBackupBeforeMaintenance is absent and a full backup exist for the database; the Maint.PrepareDbForMaintenance procedure executes a log backup. This log backup will be appended to the previous log backup file; even if in the log backup job @BkpLogsOnSameFile = 0 (Create a new backup file for each log backup). (NVarchar(512))
@SetRecoveryModeToSimple	Option to put the database in the « Simple » recovery

mode, in preparation for the maintenance.
1 = yes (put the database in the « Simple » recovery mode)
0 = no (default) recommended option.

The "Simple" recovery mode reduces the size of the log file necessary for the maintenance when you have limited disk space.

But the « Simple» recovery mode breaks the restoration sequence and a full backup will be done by the stored proc **Maint.ReturnDbToNormalUseFromMaintenance.** (int)

Samples :

A. Get exclusive access to databases GPI1, GPI2 et GPI3

This sample shows how to get exclusive access to databases **GPI1, GPI2, GPI3**.

All connections will be closed on databases **GPI1, GPI2, GPI3**.

They will be renamed to **GPI1_Maintenance, GPI2_Maintenance, GPI3_Maintenance**.

A backup of these will be put in **G:\Sql\backups**, under the names **GPI1_Maintenance.bak, GPI2_Maintenance.bak, GPI3_Maintenance.bak**.

```
Exec YourSQLDb.Maint.PrepareDbForMaintenanceMode
  @DbList =      -- Each database must be listed on its own line
  '
  GPI1
  GPI2
  GPI3
  '
  , @DbNameSuffixForMaintenance= 'Maintenance'
  , @PathOfBackupBeforeMaintenance = 'G:\Sql\backups'
```

B. Get exclusive access to databases GPI1, GPI2, and don't change the recovery mode

This sample shows how to get exclusive access to databases **GPI1, GPI2**.

All connections will be closed on databases **GPI1, GPI2**.

They will be renamed to **GPI1_Maintenance, GPI2_Maintenance**.

A backup of these will be put in **G:\Sql\backups**, under the names **GPI1_Maintenance.bak, GPI2_Maintenance.bak**.

The recovery mode will not be changed.

```
Exec YourSQLDb.Maint.PrepareDbForMaintenanceMode
  @DbList =      -- Each database must be listed on its own line
  '
  GPI1
  GPI2
  '
  , @DbNameSuffixForMaintenance = 'Maintenance'
  , @PathOfBackupBeforeMaintenance = 'G:\Sql\backups'
  , @SetRecoveryModeToSimple = 0
```

Maint.RestoreDbAtStartOfMaintenanceMode

June 25, 2014 6:07 PM

Purpose: This procedure restores the selected databases in the state they were just after the call to [Maint.PrepareDbForMaintenanceMode](#).

Upon completion, the database state is the same as it was immediately after the call to [Maint.PrepareDbForMaintenanceMode](#). This allows another Maintenance attempt. It also allows postponing of the maintenance process by making the database available to user with a call to **Maint.ReturnDbToNormalUseFromMaintenanceMode**.

Usage notes:

This procedure requires that the [Maint.PrepareDbForMaintenanceMode](#) procedure has been executed.

Its parameters must match those specified by [Maint.PrepareDbForMaintenanceMode](#). The **@DbList** parameter can contain a subset of the database set specified for [Maint.PrepareDbForMaintenanceMode](#).

Parameter name	parameter description
@DbList	A list of one or many database names, see sample below to see how to specify multiple names. The database list can be a subset of the corresponding list passed to Maint.PrepareDbForMaintenanceMode . NVarchar(max)
@DbNameSuffixForMaintenanceProcess	Suffix to add to the database names to rename them. NVarchar(128)
@PathOfBackupBeforeMaintenance	If full backups have been produced by the Maint.PrepareDbForMaintenanceMode procedure; specify here the path of the directory for those full backups. (Optional parameter). The path for the full backups will be used to return the databases to their previous state. This is useful in the case of an maintenance failure. Note that each full backup file name includes the database name, followed by the chosen suffix and the ".BAK" extension. When the @PathOfBackupBeforeMaintenance is absent and full backups exist for the databases; the RestoreDbAtStartOfMaintenanceMode procedure executes a restore of those full backups and restores all log backups up to the moment when the Maint.PrepareDbForMaintenanceMode procedure has been executed. NVarchar(512)

Sample :

This sample restore among databases renamed and backuped by [Maint.PrepareDbForMaintenanceMode](#) databases specified in @DbList which are in this sample 'GPI1, GPI2'. They are returned to their state as they were immediately after the call at [Maint.PrepareDbForMaintenanceMode](#). The parameter @DbList can be a subset of the databases processed by [Maint.PrepareDbForMaintenanceMode](#) if it is not necessary to restore all of them.

```
Exec YourSQLDb.Maint.RestoreDbAtStartOfMaintenanceMode
    @DbList = -- Each database must be listed on its own line
    '
    GPI1
    GPI2
    '
    , @DbNameSuffixForMaintenance= 'Maintenance'
    , @PathOfBackupBeforeMaintenance = 'G:\Sql\backups'
```

Maint.ReturnDbToNormalUseFromMaintenanceMode

Purpose : Rename databases to their original names from the names given by «PrepareDbForMaintenanceMode». If the database is part of YourSQLDb full backup plan, a new full backup and log backup are performed on a new file sequence. Latest backup made before maintenance are kept intact. YourSQLDb's stored procedure **Maint.SaveDbOnNewFileSet** is used to perform the new backup.

Usage notes :

Parameters must match those used for Maint.PrepareDbForMaintenanceMode. Parameter @DbList can be a subset of the one given to **Maint.PrepareDbForMaintenanceMode**.

Parameter name	parameter description
@DbList	One or many databases name, see sample below to see how to specify multiple names. Could be a subset of the corresponding list passed to PrepareDbForMaintenanceMode . These are the original names of the database before rename by PrepareDbForMaintenanceMode . (NVarChar(max))
@DbNameSuffixForMaintenance	Suffix used to rename the database. (NVarChar(128))

Sample :

This sample takes databases renamed and backup by Maint.PrepareDbForMaintenanceMode, and renames them to their original name. Databases specified in @DbList (in this sample only 'GPI1, GPI2') can be a subset of the ones specified to PrepareDbForMaintenanceMode.

```
Exec YourSQLDb.Maint.ReturnDbToNormalUseFromMaintenanceMode
    @DbList = -- Each database must be listed on its own line
    '
    GPI1
    GPI2
    '
    , @DbNameSuffixForMaintenance = 'Maintenance'
```


Maint.ReturnDbToNormalUseFromMaintenanceMode

June 27, 2014 11:19 AM

Purpose : Rename databases to their original names from the names given by «PrepDbForMaintenanceMode». If the database is part of YourSqlDb's full backup plan, a new full backup and log backup are performed on a new file sequence. Latest backups before Maintenance are kept intact. YourSqlDb's stored procedure **Maint.SaveDbOnNewFileSet** is used to perform the new backup.

Usage notes :

Parameters must match those used for Maint.PrepareDbForMaintenanceMode. Parameter @DbList can be a subset of the one given to **Maint.PrepareDbForMaintenanceMode**.

Parameter name	parameter description
@DbList	One or many databases name, see sample below to see how to specify multiple names. Could be a subset of the corresponding list passed to PrepDbForMaintenanceMode . These are the original names of the database before rename by PrepDbForMaintenanceMode . (NVarchar(max))
@DbNameSuffixForMaintenanceProcess	Suffix used to rename the database. (NVarchar(128))

Sample :

This sample takes databases renamed and taken in backup by Maint.PrepareDbForMaintenanceMode, and renames them to their original name. Databases specified in @DbList (in this sample only 'GPI1, GPI2') can be a subset of the ones specified to PrepDbForMaintenanceMode.

```
Exec YourSqlDb.Maint.ReturnDbToNormalUseFromMaintenanceMode
  @DbList = -- Each database must be listed on its own line
  ,
  GPI1
  GPI2
  ,
  , @DbNameSuffixForMaintenance= 'Maintenance'
```

Diagnostics (maintenance, performance)

June 27, 2014 2:54 PM

YourSqlDba offers comprehensive tools for maintenance diagnostics. It generates a detailed log of all maintenance queries executed, including their completion status and any error messages. The output is stored in `Maint.JobHistory`, `Maint.JobHistoryDetails`, and `Maint.JobHistoryDetailsLines`.

A more practical view of this content is provided through [Maint.HistoryView](#), which displays all queries and messages generated by **YourSqlDb**a. This view includes numerous examples of filtering results, making it easier to diagnose issues or review the actions performed during successful operations.

Common problems

1. After a new installation of YourSqlDb a you may received theses messages:

- a. An error message stating that a log backup on a given database cannot be performed until a new full backup or differential backup is completed.

This is expected behavior, as it is a requirement of SQL Server. The error will resolve once full maintenance is performed and a full backup is taken. Alternatively, you can initiate a full maintenance task or extract the full maintenance step query and adjust its parameters to perform only full backups.

- b. An error message saying some databases are not in full recovery mode.

By default the maintenance engine ensure that all databases must be in full recovery mode, which allows regular log backups through the day and then the possibility to do point-in-time restore. This is a thing which is very valuable for production databases. However if you have test databases on the same server you would wish to exclude them from this verification. See procedure [Maint>YourSqlDb DoMaint](#) in SP Reference section. Check for parameter `@ExcDbFromPolicy_CheckFullRecoveryModel` to exclude test databases from this check.

Maintenance diagnostics

To consult these tables we provide the procedure [Maint.HistoryView](#) which display all queries and messages issued by YourSqlDb a. This function has many useful examples of filtering results to ease diagnostics or survey what is done of a successful operation.

When YourSqlDb a completes successfully or not its maintenance it sends an email with the appropriate call to [Maint.HistoryView](#) with appropriate parameters (see explained examples) to display entries specific for that job. It is easier cut and paste the procedure call this way to pinpoint job details. In a error context the same call is provide with the parameter to limit display to errors shown in maintenance.

Database Mail diagnostics

Maint.DiagDbMail queries 3 Msdb tables related to Database mail diagnostics. This is an efficient way to find out why YoursqlDb alerts or messages doesn't go through. See Reference section.

Performance diagnostics

Another goodie are the performance monitoring procedures and functions. They are based on Paul Randal's queries to monitor SQL Server wait states: PerfMon.ResetAnalyzeWaitStats reset wait stats accumulated since last call of PerfMon.AnalyzeWaitStats. See PerfMon.AnalyzeWaitStats for more info on how to use it. See reference section. See Reference section.

Maint.HistoryView (V 7.0+)

July 14, 2021 4:16 PM

Purpose : Display the job maintenance history and replaces Maint.ShowHistory and Maint.ShowHistoryErrors.

Output is a standard, result set from, since Maint.HistoryView is a function (inline table view function). So output can also be further filtered by any Select statement criteria.

Base parameters are start and end datetime to get events between

Reporting is now based on time range! This means that you can now see actions of multiple parallel jobs in chronological order. JobNo column discriminates from which job the output is issued.

Columns returned are (in order)

cmdStartTime	Time of the event. Job start, informative messages, SQL statement (maintenance or to prepare maintenance), errors (from SQL doing maintenance or from sql inside YourSqlDbA)
JobNo	Job from which the command is issued. Typically 2 jobs may overlaps: The Full maintenance and the log backups, each one with its own job number.
seq	Sequence of the event, relative to all events
Secs	Optional: Duration in secs (repeated for all the query)
Typ	Type of event: Job (informations on the job) Time (of the event) Inf (informative message from YourSqlDbA) Ctx (Contextual info usually maintenance step) Sql (Query) msgs (msgs emitted by SQL, like when a backup ends) Status (success of failure of a query) YSDNo (number of message from YourSqlDbA) ErrY (Error message from YourSqlDbA) ErrLog (Error internal to YourSqlDbA, not related to maintenance).
line	Line number of the even. For most type only one line and usually, many for 'SQL', 'Msgs'
Txt	Text related to the given typ. For example for 'SQL' it is lines of the query.
	<i>The columns below are only displayed when the events originate from a job switch; otherwise, they are NULL. This provides a visual indicator when events switch from one job to another.</i>
MaintJobName	YourSqlDbA job name parameters
MainSqlCommand	The command that launched the job
Who	The user under which the job runs
Prog	The program that runs the job (Sql Agent)
Host	Le windows machine from which the command originated
SqlAgentJobName	SQL Agent job, if launched from SQL Agent
JobId	Id of the SQL Agent Job
JobStart	Hour at which the job started
JobEnd	Hour at which the job Ended

Daily YourSqlDbA report message include the query to run (a Select on the function MaintHistoryView), with proper time range for parameters, to get output of the job actions .

An extra parameter (ShowErrOnly or ShowAll) give the option to see only queries that cause an error, or all the queries in the job.

Display is easy to read. By reading the SQL output, you see SQL code lines with line is the line number of theses.

YourSQLDbA.Maint.HistoryView has the following paramters

Parameter name	parameter description
@StartTime	Display data from jobs generated between @StartTime and @EndTime (Default 1901-01-01). If the log is large, it can be long to process.
@EndTime	End time of events, If NULL it means now.
@ErrOnly	When 1, it displays Job/jobs messages and job's details for which an error is recorded, when NULL it display complete events in the period @StartTime and @EndTime.

	Special constants from YourSQLDbA.Maint.MaintenanceEnums as E
E.MHV\$ShowErrOnly	To be used as the last parameter to filter events with error only
E.MHV\$ShowAll	To be used as the last parameter to display all actions in the given time

	period
	Precomputed time values from YourSQLDbA.Maint.MaintenanceEnums as E
E.MHV\$FromMidnight	Set typically starting time parameter as the current date from midnight
E.HV\$FromYesterdayMidnight	From the start of the previous day...
E.HV\$Since12Hours	From 12 hours from now.
E.MHV\$Since1Hour	From 1 hour from now. Typically to limit events since 1 hour.
E.MHV\$Since10Min	Set starting or ending time parameter as the current time minus one hour. Typically to limit events since 10 minutes
E.MHV\$Now	Set typically ending time parameter to display events up to now.

```
-- Typical query in job report in case of error.
-- Note the extra condition on JobNo to filter out events specific to a Job.
-- Check also the E.HV$ShowErrOnly parameter, and the Order By clause.
```

```
Select cmdStartTime, JobNo, seq, Typ, line, Txt, MaintJobName, MainSqlCmd, Who, Prog, Host,
SqlAgentJobName, JobId, JobStart, JobEnd
From
-- set of constants for the function below (& precomputed date range constants)
YourSQLDbA.Maint.MaintenanceEnums AS E
-- HV$Now, HV$FromMidnight, HV$FromYesterdayMidnight
-- HV$Since12Hours, HV$Since10Min, HV$Since1Hour
-- HV$ShowErrOnly, HV$ShowAll
cross apply
YourSQLDbA.Maint.HistoryView
('2024-12-20 00:40:00.750', '2024-12-20 00:40:02.520', E.HV$ShowErrOnly)
Where JobNo=10942
Order By cmdStartTime, JobNo, Seq, TypSeq, Typ, Line
```

Samples :

```
-- In the example below, you can see what has been logged in the Last 10 minutes.
-- This is very useful for monitoring YourSqlDbA's activity.
Select cmdStartTime, JobNo, seq, Typ, line, Txt, MaintJobName, MainSqlCmd, Who, Prog, Host,
SqlAgentJobName, JobId, JobStart, JobEnd
From
YourSQLDbA.Maint.MaintenanceEnums as E
-- E.MHV$ShowErrOnly, E.MHV$ShowAll, E.MHV$FromMidnight,
-- E.MHV$Since1Hour, E.MHV$Since10Min, E.MHV$Now
cross apply YourSQLDbA.Maint.HistoryView(E.MHV$Since10Min, E.MHV$Now, E.MHV$ShowAll)
Order By cmdStartTime, Seq, TypSeq, Typ, Line
```

```
-- Since this is a function return, the output is queryable.
-- You can apply filters, such as using LIKE, just as in any SELECT statement.
-- The example below displays the activity of a given job (JobNo)
-- and DBCC CHECKDB entries in chronological order,
-- excluding those with the physical_only option.
Select DbNameClean, txt
From
YourSQLDbA.Maint.MaintenanceEnums as E
CROSS APPLY YourSQLDbA.Maint.HistoryView('2022-04-29 20:39:15.870', '2022-04-29 20:39:22.660', E.MHV$ShowAll)
Where jobNo=10223 And txt like 'dbcc check%'
and txt not like '%DBCC checkDb(%) % WITH PHYSICAL_ONLY%'
Order By JobNo, Seq, typSeq, Typ, Line
```

Results	Messages
	DbNameClean ▾ txt ▾
1	MigrationCSSMV DBCC checkDb('MigrationCSSMV')

PerfMon.ResetAnalyzeWaitStats

June 27, 2014 5:38 PM

Purpose: Reset Average Wait SQL Server statistics that indicates where are main performance problem (when a intensive resource consumption occurs) due to resources limitations. Resources limitations can be due to inadequate queries. See [PerfMon.AnalyzeWaitStats](#) for more explanations.

Usage:

```
Exec YourSQLDb.PerfMon.ResetAnalyzeWaitStats
```

PerfMon.AnalyzeWaitStats

June 27, 2014 5:45 PM

Purpose: Reset Average Wait SQL Server statistics that indicates where are main performance problem (when a intensive resource consumption occurs) due to resources limitations. Resources limitations can be due to inadequate queries.

Usage:

```
SELECT * FROM YourSQLDb.PerfMon.AnalyzeWaitStats ()
```

Output : WaitType and percentage are main column to pay attention when an intensive task occurs. When no intensive task occurs, it only means relative waits not a performance problem. You must run this query repeatedly for a while during intensive tasks after an initial call to [PerfMon.ResetAnalyzeWaitStats](#). The more the time of the intensive task ran since the last reset, the more you get the average wait time. Seek advice on the net on how to interpret these results. Not all high wait stats means a performance problem. Here an output of this function during idle time.

WaitType	Wait_S	Resource_S	Signal_S	WaitCount	Percentage	AvgWait_S	AvgRes_S	AvgSig_S
CXPACKET	141.67	85.75	55.93	75541	32.42	0.0019	0.0011	0.0007
OLEDB	111.12	111.12	0.00	2275952	25.43	0.0000	0.0000	0.0000
ASYNC_IO_COMPLETION	41.21	41.21	0.01	241	9.43	0.1710	0.1710	0.0000
BACKUPBUFFER	41.03	40.84	0.19	15442	9.39	0.0027	0.0026	0.0000
PAGEIOLATCH_SH	20.71	19.55	1.16	16646	4.74	0.0012	0.0012	0.0001
BACKUIO	20.38	19.87	0.51	76399	4.66	0.0003	0.0003	0.0000
BACKUPTHREAD	12.62	12.47	0.16	7427	2.89	0.0017	0.0017	0.0000
WRITE_COMPLETION	12.00	0.09	11.91	36067	2.75	0.0003	0.0000	0.0003
LCK_M_S	7.52	7.51	0.01	44	1.72	0.1708	0.1707	0.0002
WRITELOG	7.09	6.72	0.37	20885	1.62	0.0003	0.0003	0.0000

Maint.DiagDbMail

June 25, 2014 6:02 PM

Purpose : Diagnose mail problems.

This stored procedure requires no parameter and simply produce three results sets:

- First, it lists email error log. Problems like mail server access or mail server communication are clearly listed in the description field, with a timestamp to indicate when it occurred.
- Second, it lists emails sent, which is a proof that email server sent it, but it was caught somewhere between the SQL server and the mail server (by the anti-spam engine or the antivirus).
- Third, it lists the state of the database mail engine. Is it up and running?

No parameter is required.

Sample :

```
Exec YourSQLDb.Maint.DiagDbMail
```



Managing network drives for backup purposes

June 28, 2014 11:44 AM

For some technical reason network drive mapping must be done by the database engine service when it starts.

To perform that, you must set a SQL Server Database Engine service account that have access to network location that are going to be associated with.

If it is a normal windows user that belongs to the same windows domain that access network location, just grant it appropriate access rights (full access).

If it is not the case, this account MUST BE SET through SQL Server configuration manager as it is MANDATED by Microsoft to make this account to have proper rights for SQL Server operations. It is also especially critical in case of database encryption. Seek help for complimentary procedures related to database encryption and changing SQL Server Database Engine service account.

Changing SQL Server database Engine service account requires a service restart.

Once done you can manage network drives that are going to be set by SQL Server Database Engine service using the following procedures: [Maint.CreateNetworkDrives](#), [Maint.DisconnectNetworkDrive](#) and [Maint.ListNetworkDrives](#).

Maint.CreateNetworkDrives

June 25, 2014 5:55 PM

Purpose : Immediately map drive letters to network location. Inserts the mapping into the **Maint.NetworkDrivesToSetOnStartup** table, so SQL Server will remap them at startup by executing autostart procedure **dbo.YourSqlDbAutostart_ReconnectNetworkDrive**. This is the only object that YouSqlDb puts in master

Usage notes : Map drive letters are only visible to the database engine, not to windows explorer, and can't be used in another windows session because of the fact that they are already in use by SQL Server database engine session. To succeed SQL Server database service account must be authenticable to the network resource, in windows authentication mode.

Parameter name	parameter description
@DriveLetter	Drive letter to map on a UNC network location. Example 'R:'. (NChar(2))
@unc	UNC (Universal Naming Convention) network location to be mapped. (NVarchar(255)) Example '\\sql.rd.grics.qc.ca\SQL2005Backups'.

Sample :

```
Exec YourSQLDb.Maint.CreateNetworkDrives
    @DriveLetter = 'R:' -- Drive letter to use
    , @unc = '\\MyBackupServer\backupShare' -- UNC to the shared resource
```

Maint.DisconnectNetworkDrive

June 25, 2014 5:56 PM

Purpose : Delete network mapping, and remove network mapping definition from NetworkDrives table.

Parameter name	parameter description
@DriveLetterOrUNC	Drive letter to Disconnect. Example 'R:'. (NVarchar(255))

Sample :

```
Exec YourSQLDb.Maint.DisconnectNetworkDrive
@DriveLetterOrUNC = 'R:' -- network drive definition to remove and
delete
```

Maint.ListNetworkDrives

June 25, 2014 6:05 PM

Purpose : Show existing network drive definition in **Maint.NetworkDrivesToSetOnStartup** table.

No parameters required

Sample :

```
Exec YourSQLDb.Maint.ListNetworkDrives
```

Mirroring

June 28, 2014 11:58 AM

YourSqlDbA mirroring is much more similar to standby server than real database mirroring in SQL Server. It is based on backup and restored continuously to another server call the mirror server. This has a great advantages.

1. First it proves to you that every backups you makes are valid, since they are restored.
2. It gives you a mean to have quickly a working server with very recent databases (since log backups are taken from source server every 15 minutes, and restore immediately to mirror server, there are at most 15 minutes behind your production databases.
3. Licencing model doesn't require a supplemental licence for this server if it is not used as a production server. License can be transferred temporary to this server for 30 days as long and after it needed to be reviewed to see if it matches licencing requirements.

YourSqlDbA needs linked served to communicate with the mirror server. First in needs an standard sysadmin standard account on the destination server. Then use [Mirroring.AddServer](#) to define it properly.

Next modify job step that perform backups with [Maint>YourSQLDbA DoMaint](#) stored procedure. Pay attention to parameters @mirrorServer, @ReplacePathsInDbFileNames and @ReplaceSrcBkpPathToMatchingMirrorPath. Parameter @ReplacePathsInDbFileNames is mandatory in many circumstances.

The first parameter @mirrorServer is mandatory for the jobstep that does full backup and the one that does log backups. The second is needed if your databases locations can't be the same on the source and target servers. The last is needed when backup location can be expressed in the same way from the source and the destination server. This happen when for example when you backup to a UNC share which is a directory on the mirror server, and you want to translate this UNC location into a real local directory path.

Failover of selected database is performed using [Mirroring.Failover](#) if source server is able to do it and must be always attempted, because if puts offline source databases that must not be accessible anymore. [Mirroring.DoRecovery](#) takes selected target server databases our of restoring mode this available to application. This does not the magic of changing connection strings to point to the new server. Some admin change server name instead to do the trick or change DNS entries.

Mirroring can be used as a trick to reduce maintenance windows required to achieve migration of application's databases compatibles to a newer version of SQL Server. You set up a mirror server to this newer instance version. The next night all backups full backup are going to be restore to destination server. During the next day all log backups are going to be restored every 15 minutes. This is here that [Mirroring.Failover](#) does the magic. It gets the last log backup from source server and restore it and puts source database offline. It is much quicker to get the latest version of the database. For databases in simple recovery mode it performs complete backup/restore again. Usually most production databases are in complete recovery mode and log backup restore does the trick for them. [Mirroring.Failover](#) have database selection parameters, so we suggest to restrict the list of database to production database first. Other databases migration can then be deferred to later. During the process some minimal compatibility trick can be done. One is related to fulltext catalogs disk structure that change from SQL2005 to later versions. [Mirroring.Failover](#) calls [Upgrade.MakeDbCompatibleToTarget](#) to convert them to the new structure.

Mirroring.AddServer

June 26, 2014 11:33 AM

Purpose: Add a mirror SQL instance in YourSQLDbA for the source SQL instance.

Usefulness of the mirror SQL instance

The SQL Agent jobs which uses this mirror SQL instance name in the parameter, will execute a restore of the backups to the mirror SQL instance.

The goal of this restoration is to verify the integrity of the backup files.

The mirror SQL instance will also contain a copy of the logins, to allow it to become a standby server.

The automatic restoration works like this; as soon as YourSQLDbA generates a backup file, it restores it to a mirror SQL instance. If this restoration fails, an email is sent to the SQL administrator.

Prerequisite to use this functionality:

- Both SQL instances must have the same YourSQLDbA version. You can execute **YourSQLDbA.Install.ShowAndKeepTrackOfVersionInfo** to see the YourSQLDbA version.
- Backup directories must be accessible by both SQL instances. If both SQL instances are not on the same machine, the backup directories must be network shared like '\\server\SQLBackupShare'.
- The SQL Server Agent starting account on the source instance must also exist as a « sysadmin » login on the mirror SQL instance.
- On first use, the original databases must not exist on the mirror SQL instance.
- Databases to be restored on the mirror SQL instance must ALWAYS be in the « RECOVERING » state on the mirror instance, for as long as the automatic backup restoration functionality is activated.

Workings of the "Mirroring.AddServer" procedure

The **Mirroring.AddServer** adds the mirror SQL instance to YourSQLDbA by doing the following :

- It creates a link server for the mirror SQL instance.
- It tests that YourSQLDbA versions are the same on both SQL instances.
- It adds the mirror instance name in the **Mirroring.TargetServer** table on the source instance.
- It adds the source instance name in the **Mirroring.TargetServer** table on the mirror instance.
- It executes a copy of all SQL logins to the mirror SQL instance.

Execute the global maintenance after the execution of Mirroring.AddServer

After the execution of the **Mirroring.AddServer** procedure and for each databases with a recovery model **Full** or **Bulk-logged**, you must execute the [Maint.YourSQLDbA_DoMaint](#) or [Maint.SaveDbOnNewFileSet](#) procedure to generate a full backup for each mirrored database in the mirror instance.

If there is no full backup of the database when the next log backup is executed, YourSQLDbA will send an email to the database administrators to warn that a full backup must be done before making a log backup.

The email will lead you to the following error message:

To restore a LOG backup to the mirror server the database <your database name> in the mirror server must exist and be in «RESTORING» state.

Parameter name	Parameter description
@MirrorServer	Mirror SQL instance name. (Required parameter). Example: 'MirrorServ\MirrorSQL2005' (NVarchar(255))
@RemoteLogin	SQL login of type « SQL Server authentication » in the mirror SQL instance. (Required parameter). This login must have sysadmin server role in the mirror SQL instance. You can use the 'sa' login. (NVarchar(255))
@RemotePassword	Password for the login in the the mirror SQL instance. (Required parameter). (NVarchar(255))
@YourSqlDbAccount ForMirroringPwd	(Facultative) Password of the YourSqlDb login. <u>Only required</u> when if multiple mirroring source instances of YourSqlDb share the same destination server.

Sample:

Add a mirror SQL instance in YourSQLDb for the source SQL instance

This example adds the mirror SQL instance 'MirrorServ\MirrorSQL2005' in YourSQLDb for the source SQL instance.

```
-- This command must be executed in the source instance
Exec YourSQLDb.Mirroring.AddServer
    @MirrorServer = 'MirrorServ\MirrorSQL2005'
, @RemoteLogin = 'sa'
, @RemotePassword = 'RemotePassword'
```

Mirroring.DoRecovery

June 26, 2014 11:35 AM

Purpose: Make selected databases out of restoring state in the SQL mirror server to make them accessible.

Prerequisite to execute "Mirroring.DoRecovery":

- The databases to be restored in the SQL mirror instance must be in the "(RESTORING)" state.

Execute a full backup after the recovery

After the execution of the **Mirroring.DoRecovery** procedure and for each databases with a recovery model **Full** or **Bulk-logged**, you must make a full backup of the recovered databases. On the next log backup, YourSQLDbA will send an email to the database administrators to warn that a full backup must be done before making a log backup.

The email will lead you to the following error message:

```
Log backup forbidden before doing a first full backup of [ <database name> ] status is <status of the database>.
```

Parameter name	Parameter description															
@IncDb	List of databases to be made accessible. (Optional parameter). When left empty, all databases are selected. Each database inclusion filter must be on its own line between the single quotes. The databases names in the list, can include wildcard characters from the "Select" instruction "like" clause. (NVarchar(max)) <table><tr><th>Wildcard character</th><th>Description</th><th>Example</th></tr><tr><td>%</td><td>Any string of zero or more characters.</td><td>'%base%' selects all databases names that contains the word "base" anywhere in the database name.</td></tr><tr><td>_ (underscore)</td><td>Any single character.</td><td>'an_' selects any three-letter database names beginning with the letters "an", like "An1" or "An9" or "AnF".</td></tr><tr><td>[0-9]</td><td>Any single character between 0 and 9.</td><td>'base[5-9]' selects any five-letter database names beginning with the letters "base", and followed by a number between 5 and 9, like « base5 » or « base9 ».</td></tr><tr><td>[]</td><td>The underscore character.</td><td>'%[_]test' selects any database names ending with the characters "_test". The underscore between brackets is interpreted as a character instead of a wildcard character. It is the same behaviour with '%', ']' and '['.</td></tr></table>	Wildcard character	Description	Example	%	Any string of zero or more characters.	'%base%' selects all databases names that contains the word "base" anywhere in the database name.	_ (underscore)	Any single character.	'an_' selects any three-letter database names beginning with the letters "an", like "An1" or "An9" or "AnF".	[0-9]	Any single character between 0 and 9.	'base[5-9]' selects any five-letter database names beginning with the letters "base", and followed by a number between 5 and 9, like « base5 » or « base9 ».	[]	The underscore character.	'%[_]test' selects any database names ending with the characters "_test". The underscore between brackets is interpreted as a character instead of a wildcard character. It is the same behaviour with '%', ']' and '['.
Wildcard character	Description	Example														
%	Any string of zero or more characters.	'%base%' selects all databases names that contains the word "base" anywhere in the database name.														
_ (underscore)	Any single character.	'an_' selects any three-letter database names beginning with the letters "an", like "An1" or "An9" or "AnF".														
[0-9]	Any single character between 0 and 9.	'base[5-9]' selects any five-letter database names beginning with the letters "base", and followed by a number between 5 and 9, like « base5 » or « base9 ».														
[]	The underscore character.	'%[_]test' selects any database names ending with the characters "_test". The underscore between brackets is interpreted as a character instead of a wildcard character. It is the same behaviour with '%', ']' and '['.														
@ExcDb	List of databases to exclude from the database list generated by the @IncDb parameter. (Optional parameter). When the list is empty, no database is excluded. Each database exclusion filter must be on its own line between the single quotes. The databases names in the list, can include wildcard characters from the "Select" instruction "like" clause. See the @IncDb parameter above for a description of the wildcard characters. (NVarchar(max))															

Samples:

A. Make accessible all databases that are in "RESTORING" state

The following example make's accessible all databases that are in "RESTORING" state.

```
-- This command must be executed on the SQL mirror instance.
Exec YourSQLDbA.Mirroring.DoRecovery
@IncDb = -- Each database inclusion filter must be
         -- on its own line between the following single quotes:
'
'
, @ExcDb = -- Each database exclusion filter must be
         -- on its own line between the following single quotes:
'
```

'

B. Make accessible the selected databases that are in "RESTORING" state

The following example make's accessible the selected databases that are in "RESTORING" state.

```
-- This command must be executed on the SQL mirror instance.
Exec YourSQLDb.Mirroring.DoRecovery
-- Each database inclusion filter must be
-- on its own line between the following single quotes:
@IncDb =

'
ACHAT
Sales%
'

-- Each database exclusion filter must be
-- on its own line between the following single quotes:
, @ExcDb =

'
%[_]Test
%temp[0-9]
%temp[0-9][0-9]
'
```

Mirroring.Failover

June 26, 2014 11:37 AM

Purpose: This procedure performs a Failover to the standby server of the selected databases. Used in a upgrade process, for which the target instance is at a newer SQL Server version, it calls procedure Upgrade.MakeDbCompatibleToTarget. This procedure set compatibility level of the database to SQL Server destination version. If the database is SQL2005, it upgrades fulltext catalogs to the new structure of higher version. Databases selected on source instance are put offline as a mean to protect them from being accessed by an application and being modified instead of the one on the failover server.

Prerequisites to execute " Mirroring.Failover ":

- Having a working mirroring server.

Workings of the " Mirroring.Failover " procedure

The **Snapshot** databases in the source server are excluded automatically from the list of selected databases.

The **Mirroring.Failover** procedure executes the following steps for each selected database:

1. It synchronizes the SQL connection codes from the source server to the mirror server.
2. It checks that the database is **Online** in the source server.
3. It checks if database to restore is not there, or there in restoring state on the mirror server.
4. It checks that the database has been configured to be restored in a mirror server.
5. It checks destination server version.
6. It closes existing connections to the database and puts the database OFFLINE and then ONLINE to cut existing connexions in the source server.
7. If the recovery model of the database is not **Simple**, the procedure executes a last log backup in the source server.
8. If the recovery model of the database is **Simple**, the procedure executes a differential backup of the database in the source server.
9. It puts the database (**OffLine**) in the source server.
10. It restores the database's log, differential on the destination server and makes the database accessible.
11. It sets back database ownership to the original login
12. It changes the compatibility level of the database in the mirror server to match instance version.

When all databases have been processed, the **Mirroring.Failover** procedure shows the migration status of each database.

You must perform a full backup of migrated databases on the mirror instance for databases that are not in simple recovery model.

After the execution of the **Mirroring.Failover** procedure and for each databases with a recovery model **Full** or **Bulk-logged**, you must execute the [Maint.YourSQLDb](#) or [Maint.SaveDbOnNewFileSet](#) procedure to generate a full backup for each migrated database in the mirror server.

If there is no full backup of the database when the next log backup is executed, YourSQLDb will send an email to the database administrators to warn that a full backup must be done before making a log backup of theses databases.

The email will lead you to the following error message:

Log backup forbidden before doing a first full backup of [<database name>] status is <status of the database>.

Parameter name	Parameter description												
@Simulation	Simulation mode to get potential error messages. 1 = yes. 0 = no (défault). (Int)												
@IncDb	List of the databases to be processed. (Optional parameter). When left empty, all databases are selected. Each database inclusion filter must be on its own line between the single quotes. The databases names in the list, can include wildcard characters from the "Select" instruction "like" clause. (NVarChar(max)) <table><tr><th>Wildcard character</th><th>Description</th><th>Example</th></tr><tr><td>%</td><td>Any string of zero or more characters.</td><td>'%base%' selects all databases names that contains the word "base" anywhere in the database name.</td></tr><tr><td>_ (underscore)</td><td>Any single character.</td><td>'an_' selects any three-letter database names beginning with the letters "an", like "An1" or "An9" or "AnF".</td></tr><tr><td>[0-9]</td><td>Any single character between 0 and 9.</td><td>'base[5-9]' selects any five-letter database names beginning with the letters "base", and followed by a number between 5 and 9, like « base5 » or « base9 ».</td></tr></table>	Wildcard character	Description	Example	%	Any string of zero or more characters.	'%base%' selects all databases names that contains the word "base" anywhere in the database name.	_ (underscore)	Any single character.	'an_' selects any three-letter database names beginning with the letters "an", like "An1" or "An9" or "AnF".	[0-9]	Any single character between 0 and 9.	'base[5-9]' selects any five-letter database names beginning with the letters "base", and followed by a number between 5 and 9, like « base5 » or « base9 ».
Wildcard character	Description	Example											
%	Any string of zero or more characters.	'%base%' selects all databases names that contains the word "base" anywhere in the database name.											
_ (underscore)	Any single character.	'an_' selects any three-letter database names beginning with the letters "an", like "An1" or "An9" or "AnF".											
[0-9]	Any single character between 0 and 9.	'base[5-9]' selects any five-letter database names beginning with the letters "base", and followed by a number between 5 and 9, like « base5 » or « base9 ».											

	[_]	The underscore character.	'%[_]test' selects any database names ending with the characters "_test". The underscore between brackets is interpreted as a character instead of a wildcard character. It is the same behaviour with '%', ']' and '['.
@ExcDb	List of databases to exclude from the database list generated by the @IncDb parameter. (Optional parameter). When the list is empty, no database is excluded. Each database exclusion filter must be on its own line between the single quotes. The databases names in the list, can include wildcard characters from the "Select" instruction "like" clause. (NVarChar(max)) See the @IncDb parameter above for a description of the wildcard characters. (NVarChar(max))		
@lastDataSync	Controls if last backup/restore to sync data are going to be performed at Fail Over. 1 by default. Set to 0 to prevent synching. Useful for third party stand-by server solution like CommVault SQL Live Sync.		

Samples:

A. Migrate all databases to SQL Server 2008

The following example migrates all databases to SQL Server 2008.

```
-- This command must be executed on the SQL source instance.
Exec YourSQLDb.Mirroring.Failover
    @Simulation = 0
-- Each database inclusion filter must be
-- on its own line between the following single quotes: '
, @IncDb =
'
'
-- Each database exclusion filter must be
-- on its own line between the following single quotes: '
, @ExcDb =
'
'
```

B. Migrate the selected databases to SQL Server 2008

The following example migrates the selected databases to SQL Server 2008.

```
-- This command must be executed on the SQL source instance.

Exec YourSQLDb.Mirroring.Failover
    @Simulation = 0
-- Each database inclusion filter must be
-- on its own line between the following single quotes: '
, @IncDb =
'
ACHAT
Sales%
'
-- Each database exclusion filter must be
-- on its own line between the following single quotes: '
, @ExcDb =
'
%[_]Test
%temp[0-9]
%temp[0-9][0-9]
'
```

Upgrade.MakeDbCompatibleToTarget

June 26, 2014 11:39 AM

Purpose: Set compatibility level to match target instance version. If source database is SQL2005, rebuild catalog structure accordingly to higher SQL version.

The " Upgrade. MakeDbCompatibleToTarget " procedure executes the following steps:

1. It changes the compatibility level of the database in the mirror server to match instance version.
2. It checks that the destination server is higher than **SQL Server 2005** server, if it is the case:
 - a. It puts the database in **MULTI_USER** mode.
 - b. It creates a new FILEGROUP which contains all full text search indexes of the database.
 - c. It populates the new indexes to benefit from the new full text search advances provided by newer SQL Server versions higher than **SQL Server 2005**.

Parameter name	Parameter description
@DbName	Name of the database to convert. Mandatory. (sysname)

Example :

Finalize the manual migration (by backup restore) of a database

The following example finalize the migration of the database **Database_to_be_converted** to the SQL Server 2008 format.

```
Exec YourSQLDb.Upgrade.MakeDbCompatibleToTarget  
@DbName = 'Database_to_be_converted'
```

Mirroring.SetYourSqlDbAccountForMirroring

March 7, 2016 3:09 PM

Purpose: Set a common password for YourSqlDb account when multiple server share a single mirror.

Prerequisite to execute

- It changes linked server configurations used for mirroring. Some Mirror need to be defined.

Parameter name	Parameter description
@YourSqlDbAccountForMirroringPwd	A common password to set for multiple source servers sharing a single mirror. If no value is supplied, and a connection can't be made, it resets the password to a random generated value.

Set YourSqlDb account password:

The following example make's accessible all databases that are in "RESTORING" state.

```
-- This command must be executed on the SQL mirror instance.  
Exec YourSQLDb.Mirroring.SetYourSqlDbAccountForMirroring  
@YourSqlDbAccountForMirroringPwd='YourSecretPasswordSauce'  
'
```